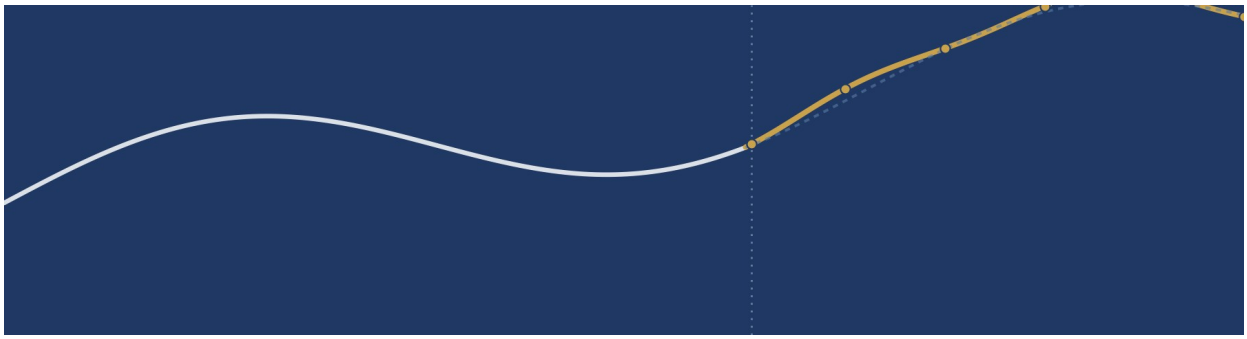


FORECASTING WITH FOUNDATION & TRANSFORMER MODELS

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

INTERNSHIP REPORT

Internship Track

Artificial Intelligence and Machine Learning

PROJECT TITLE

Implementation of PatchTST and Chronos Model

UNDER THE GUIDANCE OF

Dr. Rashmi Dutta Baruah

COMPLETED BY

Twinkle Dwivedi

Department of Computer Science and Engineering

TECHNICAL SUMMARY

Time Series Forecasting on Wind Power Generation Data

1 Chronos Model (Chronos.ipynb)

Aim. Demonstrate and evaluate the performance of the Chronos model (amazon/chronos-t5-small) for zero-shot time series forecasting on wind power generation data, assessing forecasting effectiveness without any task-specific training via Mean Squared Error (MSE), Mean Absolute Error (MAE), and inference latency.

Dataset. Wind power generation data from multiple .csv files in the /content/Dataset directory. Each file represents a different location with Time (datetime) and Power (target) columns; the pipeline auto-detects these columns, combines the series into a unified DataFrame, and filters for series long enough to support the defined forecast horizons — ultimately loading 4 distinct series.

Experimental Analysis. A rolling evaluation across forecast horizons of 24, 48, 96, 192, and 336 steps. For each horizon, data is split into historical context and future actuals using a TEST_FRAC of 20%. The Chronos pipeline performs probabilistic predictions, taking the median of 20 samples as the point forecast, and handles predictions longer than the model's recommended MAX_PREDICTION_LENGTH_RECOMMENDED (64) via chunked predictions. MSE, MAE, and inference latency are recorded for each horizon.

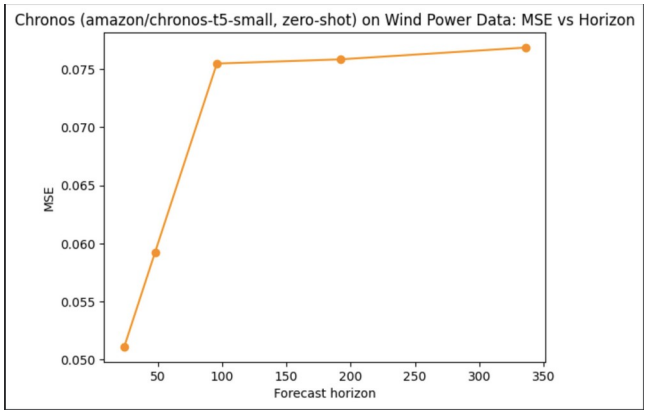


Figure 1. Chronos — MSE vs. Forecast Horizon

✦ **KEY RESULT**

Results are consolidated in results_chronos_wind.csv and visualized as MSE vs. forecast horizon. As the forecast horizon increases, both MSE and MAE tend to rise, indicating longer-term predictions are more challenging. MSE for a 24-step horizon is approximately 0.051, rising to about 0.076–0.077 for horizons of 192 and 336 steps. Inference latency also increases with horizon length due to the larger number of prediction steps and chunking operations.

2 PatchTST Model (PatchTST.ipynb)

Aim. Demonstrate the performance of the PatchTST model on wind power generation data for multi-horizon forecasting, comparing performance under two data availability scenarios — the full dataset versus a significantly smaller (10%) portion of training data — across various forecast horizons.

Dataset. Wind power generation data (likely sourced from Kaggle), loaded from multiple CSV files in /content/Dataset. The pipeline automatically identifies datetime and power-output columns, standardizes them into ds (datetime), y (target power), and unique_id (per location/turbine) fields, infers an hourly sampling frequency, and filters out series with missing columns or insufficient length.

Experimental Analysis. A single, global PatchTST model is trained across all wind-turbine series, iterating through forecast horizons of 24, 48, 96, 192, and 336 hours and training-data fractions of 1.0 (full data) and 0.10 (low data). For each combination, the last 20% of each series is reserved for testing, and a rolling-window evaluation computes MSE and MAE. Training time, inference latency, and model parameters are also tracked.

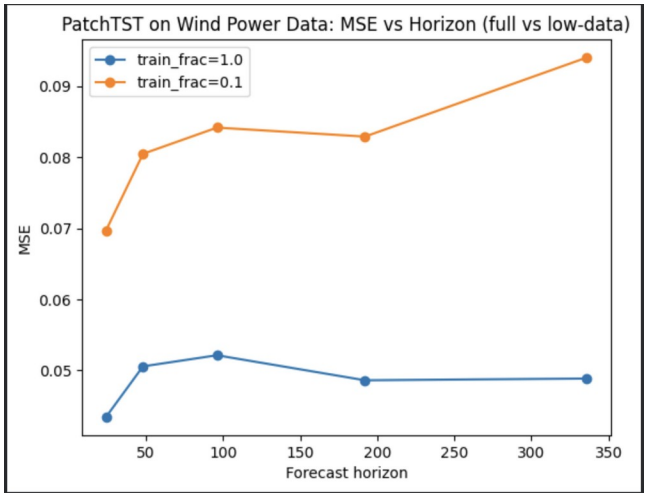


Figure 2. PatchTST — MSE vs. Horizon (full-data vs. low-data)

✦ **KEY RESULT**

Results are consolidated in results_patchtst_wind.csv. MSE generally increases as the forecast horizon grows, indicating longer-term predictions are more challenging. The model trained with the full dataset (train_frac = 1.0) consistently achieves a significantly lower MSE than the model trained on only 10% of the data (train_frac = 0.1), underscoring the importance of data volume for forecasting accuracy — confirmed visually by the full-data curve sitting well below the low-data curve.

3 Data Efficiency Comparison — PatchTST and Chronos

(PatchTST and Chronos.ipynb)

Aim. Conduct a data efficiency comparison between PatchTST and Chronos, evaluating how each model performs under varying data availability and across different forecast horizons — training PatchTST on full and reduced datasets, and testing Chronos in zero-shot settings with full and short context lengths — to understand robustness and predictive accuracy.

Experimental Analysis. Time series data is loaded from four CSV files (Location1–4.csv) with Time as datetime and Power as the target, processed to infer an hourly sampling frequency. A consistent testing methodology holds out the last 20% of each series, and a shared test-window builder ensures both models are evaluated on the same forecast origins through rolling-window predictions. MSE, MAE, training time, and inference latency are collected throughout.

Comparison Setup. For PatchTST, training on 100% versus 10% of the available data is compared. For Chronos, a zero-shot approach compares a full context window (512 points) against a short context window (51 points, ~10% of full context). The full sweep runs across five forecast horizons: 24, 48, 96, 192, and 336 hours.

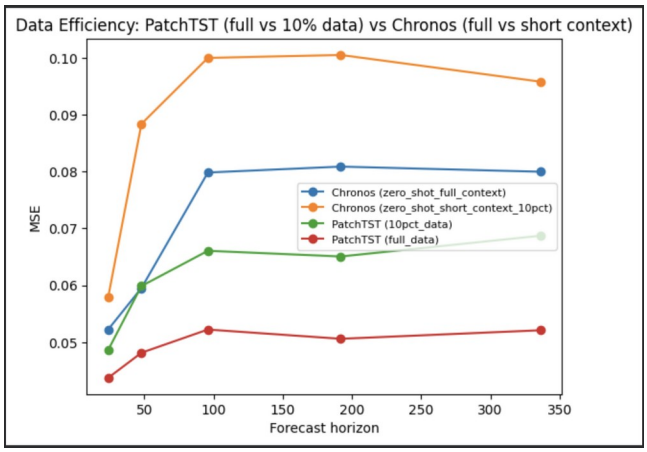


Figure 3. Data Efficiency — PatchTST (full vs. 10% data) vs. Chronos (full vs. short context)

✦ **KEY RESULT**

Results are consolidated in results_data_efficiency_comparison.csv. PatchTST with full data generally achieves the lowest MSE overall; PatchTST with 10% data outperforms Chronos with a short context. Chronos with full context performs reasonably well but degrades more sharply at longer horizons compared to PatchTST. As a zero-shot model, Chronos requires no training time, but shows higher inference latency and significantly more parameters than PatchTST — highlighting trade-offs between model complexity, data efficiency, and performance.

SUMMARY

At a Glance — Model Comparison

Aspect	Chronos (zero-shot)	PatchTST (full data)	PatchTST (10% data)
Training required	None (zero-shot)	Full training run	Reduced training run
Typical MSE range	0.051 – 0.077	0.043 – 0.052	0.070 – 0.095
Accuracy at long horizon	Degrades noticeably	Most stable	Degrades most
Inference latency	Higher	Lower	Lower
Best suited for	Quick, data-scarce use	Highest-accuracy needs	Fast, low-data prototyping

✦ TAKEAWAY

Across all three notebooks, model accuracy consistently declines as the forecast horizon lengthens, and data volume matters: PatchTST trained on the full dataset is the strongest and most stable performer, while Chronos's zero-shot capability offers a compelling accuracy/effort trade-off when little or no training data is available.